Laravel backend kódmagyarázó segédlet

Route, controller, model, migration, validáció, API és adatbázis összekötések felépítése példakódokkal.

1. Laravel backend felépítése

Laravelben a backend fő részei: route, controller, model, migration, request validation és database kapcsolat. A route megmondja, milyen URL-re mi történjen. A controller tartalmazza a műveletet. A model az adatbázistáblát képviseli. A migration létrehozza a táblát.

Elem	Feladata	Példa
Route	URL és metódus összekötése	GET /api/products
Controller	Logika futtatása	ProductController@index
Model	Adatbázis rekord kezelése	Product::all()
Migration	Tábla szerkezete	products tábla
Request	Validáció	name required, price numeric

2. Projekt létrehozás és alap parancsok

Várt eredmény: http://127.0.0.1:8000 címen fut a Laravel. Ha composer nem ismert, Composer nincs telepítve vagy nincs PATH-ban.

```
# Laravel projekt létrehozása
composer create-project laravel/laravel backend

# Belépés a mappába
cd backend

# .env létrehozás, ha kell
copy .env.example .env

# Alkalmazás kulcs
php artisan key:generate

# Fejlesztői szerver
php artisan serve
```

3. Adatbázis beállítás .env fájlban

XAMPP/AMPPS esetén gyakran root felhasználó és üres jelszó van. Ha más adatbázisnevet használsz, a DB_DATABASE értéke is legyen ugyanaz.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=projekt_neve
DB_USERNAME=root
DB_PASSWORD=
```

4. Model, migration és controller egyszerre

Ha egy új adattípust kell létrehozni, például termék, ingatlan, kategória, túra, felhasználó, akkor gyakran ez a leggyorsabb parancs.

-m = migration, -c = controller, -r = resource controller. Így egyszerre megkapod a fő fájlokat.

```
php artisan make:model Product -mcr
```

5. Migration példa

A migration mondja meg, milyen oszlopok legyenek az adatbázisban. A mezők típusa igazodjon a feladathoz: szöveg string/text, szám integer/decimal, logikai érték boolean, dátum date/dateTime.

> Ha a feladat más mezőket kér, csak az oszlopokat cseréled. Kapcsolatnál foreignId kell.

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description')->nullable();
        $table->integer('price');
        $table->boolean('active')->default(true);
        $table->foreignId('category_id')
              ->constrained()
              ->cascadeOnDelete();
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('products');
}
```

6. Migration futtatás

> Ha table already exists hibát kapsz, vagy már létezik a tábla, vagy rossz adatbázisban vagy. Fejlesztésnél migrate:fresh gyors megoldás, de törli az adatokat.

```
# Migrationök futtatása
php artisan migrate

# Ha mindent újra akarsz építeni
php artisan migrate:fresh

# Ha seedert is futtatnál
php artisan migrate:fresh --seed
```

7. Model beállítás

A modelben meg kell adni, mely mezők tölthetők tömegesen. Ez a $fillable. Kapcsolatoknál belongsTo, hasMany, hasOne metódusokat írsz.

> Ha MassAssignmentException hibát kapsz, hiányzik a mező a $fillable tömbből.

```
class Product extends Model
{
    protected $fillable = [
        'name',
        'description',
        'price',
        'active',
        'category_id'
    ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

8. Kapcsolódó modellek

> Egy kategóriának több terméke lehet: hasMany. Egy termék egy kategóriához tartozik: belongsTo.

```
class Category extends Model
{
```

```
    protected $fillable = ['name'];

    public function products()
    {
        return $this->hasMany(Product::class);
    }
}
```

9. API route-ok

API végpontokat routes/api.php fájlba írj. Resource controller esetén egy sorral létrejön az index, store, show, update, destroy útvonal.

> Ez létrehozza például: GET /api/products, POST /api/products, GET /api/products/{id}, PUT/PATCH /api/products/{id}, DELETE /api/products/{id}.

```
use App\Http\Controllers\ProductController;

Route::apiResource('products', ProductController::class);
```

10. Controller - lista lekérése

> with('category') akkor kell, ha a kapcsolt kategória adatait is vissza akarod küldeni. Enélkül csak category_id lesz.

```
public function index()
{
    $products = Product::with('category')
        ->orderBy('name')
        ->get();

    return response()->json($products);
}
```

11. Controller - új adat mentése

> A validate rész védi a backend oldalt. Ha nem jó adat jön frontendről, Laravel 422-es hibával visszadobja.

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'nullable|string',
        'price' => 'required|integer|min:0',
        'active' => 'boolean',
        'category_id' => 'required|exists:categories,id'
    ]);

    $product = Product::create($validated);

    return response()->json($product, 201);
}
```

12. Controller - egy rekord lekérése

> A Product $product automatikus model binding. Az URL-ben kapott id alapján Laravel megkeresi a rekordot.

```
public function show(Product $product)
{
    $product->load('category');

    return response()->json($product);
}
```

13. Controller - módosítás

> Update-nél használható a sometimes, mert nem biztos, hogy minden mező jön.

```
public function update(Request $request, Product $product)
```

```
{
    $validated = $request->validate([
        'name' => 'sometimes|required|string|max:255',
        'description' => 'nullable|string',
        'price' => 'sometimes|required|integer|min:0',
        'active' => 'boolean',
        'category_id' => 'sometimes|required|exists:categories,id'
    ]);

    $product->update($validated);

    return response()->json($product);
}
```

14. Controller - törlés

> Ha kapcsolt rekordok vannak, figyelj a migrationben a cascadeOnDelete vagy restrict viselkedésre.

```
public function destroy(Product $product)
{
    $product->delete();

    return response()->json([
        'message' => 'Sikeres törlés'
    ]);
}
```

15. Seeder - tesztadatok

```
php artisan make:seeder ProductSeeder
```

Seeder tartalma

```
public function run(): void
{
    $category = Category::create([
        'name' => 'Elektronika'
    ]);

    Product::create([
        'name' => 'Laptop',
        'description' => 'Használt üzleti laptop',
        'price' => 180000,
        'active' => true,
        'category_id' => $category->id
    ]);
}
```

DatabaseSeeder

> Seeder akkor hasznos, ha gyorsan kell próbaadat, és nem akarsz kézzel adatbázisba írni.

```
public function run(): void
{
    $this->call([
        ProductSeeder::class,
    ]);
}
```

16. Frontend összekötés API-val

A backend akkor van jól összerakva, ha a böngészőből vagy Postmanből a /api/products JSON-t ad vissza. Ezután Vue fetch vagy axios hívással tudja használni.

> Ha CORS hiba van, a frontend eléri az URL-t, de a böngésző blokkolja. Ilyenkor Laravel CORS beállítást kell ellenőrizni.

```
// Vue oldali példa
const response = await fetch('http://127.0.0.1:8000/api/products')
const products = await response.json()
```

17. Gyakori Laravel hibák és javítások

Hiba	Ok	Javítás
Class not found	Nincs import vagy rossz namespace	use App\Models\Product;
Base table not found	Nem futott migration	php artisan migrate
Unknown database	Rossz DB_DATABASE	.env javítás, adatbázis létrehozás
MassAssignmentException	Hiányzik fillable	Modelben $fillable bővítése
419 error	Web route CSRF gond	API-hoz api.php és JSON kérés
422 error	Validációs hiba	Mezőnevek és értékek ellenőrzése
500 error	Backend kivétel	storage/logs/laravel.log megnézése

18. Optimalizált feladatmegoldási sorrend

Lépés	Mit csinálj?
1	Írd fel az entitásokat: például Product, Category, User
2	Döntsd el a mezőket és adattípusokat
3	Hozd létre model + migration + controller fájlokat
4	Írd meg a migrationöket
5	Futtasd a migrationt
6	Modelben állítsd be fillable és kapcsolatokat
7	Route-ok létrehozása
8	Controller metódusok: index, store, show, update, destroy
9	Teszteld böngészővel/Postmannel
10	Csatlakoztasd Vue vagy Blade frontendhez